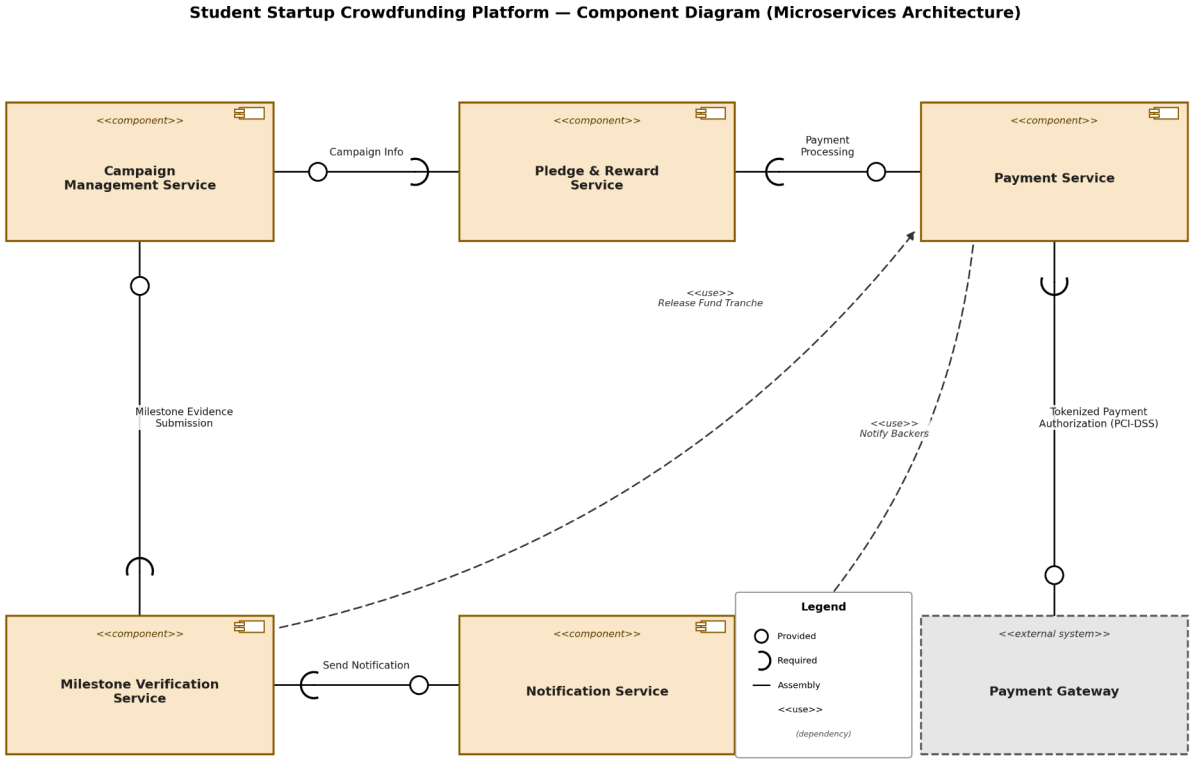
Lab 3: Component Modelling & Architectural Pattern Selection

Architectural Justification — Student Startup Crowdfunding Platform

Name: B Sanjana SRN: PES2UG24CS106 Section: B Course: UE24CS341A

Problem Statement #40 — Retail, E-Commerce & Finance: Student Startup Crowdfunding Platform

Component Diagram:



Architecture Selection: We chose Microservices Architecture for the Student Startup Crowdfunding Platform system.

Justification

Reason 1 — Independent scaling for heterogeneous workloads: The platform has functionally distinct, independently-loaded responsibilities: campaign creation and browsing (Campaign Management), pledge and reward-tier handling (Pledge & Reward Service), payment authorization (Payment Service), milestone review (Milestone Verification Service), and backer alerts (Notification Service). Pledging and payment traffic spikes sharply during active campaign launches while milestone review and notification traffic is comparatively steady. Microservices let each service be scaled independently to match its own demand instead of over-provisioning the entire system, which a monolithic layered design would require.

Reason 2 — Fault isolation and independent deployment for milestone-gated releases: FR-001 requires that funds be released only in tranches after Faculty Committee verification, and FR-004/FR-005 require evidence review and backer notification to happen reliably around that gate. Isolating Milestone Verification, Payment, and Notification as separate services means a fault or slow dependency in one (e.g., the Notification Service) cannot block fund disbursement or pledge collection. Each service can also be developed, tested, and redeployed independently — e.g., updating reward-tier logic in the Pledge & Reward Service does not require redeploying the Payment Service.

Security Advantage: NFR-001 requires PCI-DSS compliance with tokenized payment authorizations. Microservices allow the Payment Service to be isolated as its own deployable unit with its own restricted network boundary and access controls, so PII and cardholder-data handling is confined to one auditable component rather than being embedded inside a shared business layer. This narrows the PCI-DSS compliance scope to a single service instead of the whole application, and limits the impact of a breach in any other service (e.g., Notification Service) since it never has direct access to raw payment data — only the Payment Service exchanges tokenized data with the external Payment Gateway.

Performance Benefit: Because each microservice is deployed and scaled independently, the Pledge & Reward Service and Payment Service — the components under the heaviest concurrent load during active fundraising — can be horizontally scaled with additional instances without needing to scale the Milestone Verification or Notification services, which see comparatively low, bursty traffic. This targeted scaling keeps pledge-to-payment latency low during peak campaign activity while avoiding the resource waste of scaling an entire monolithic/layered application uniformly, and it satisfies the ≥99.5% platform uptime target in NFR-002 since a slowdown in one service does not directly degrade the others.

Note: The accompanying component diagram shows five components — Campaign Management Service, Pledge & Reward Service, Payment Service, Milestone Verification Service, and Notification Service — with provided/required (ball-and-socket) interfaces for Campaign Info, Payment Processing, Milestone Evidence Submission, Send Notification, and Tokenized Payment Authorization (to the external Payment Gateway), plus <<use>> dependencies showing the fund-tranche release and backer-notification flows.